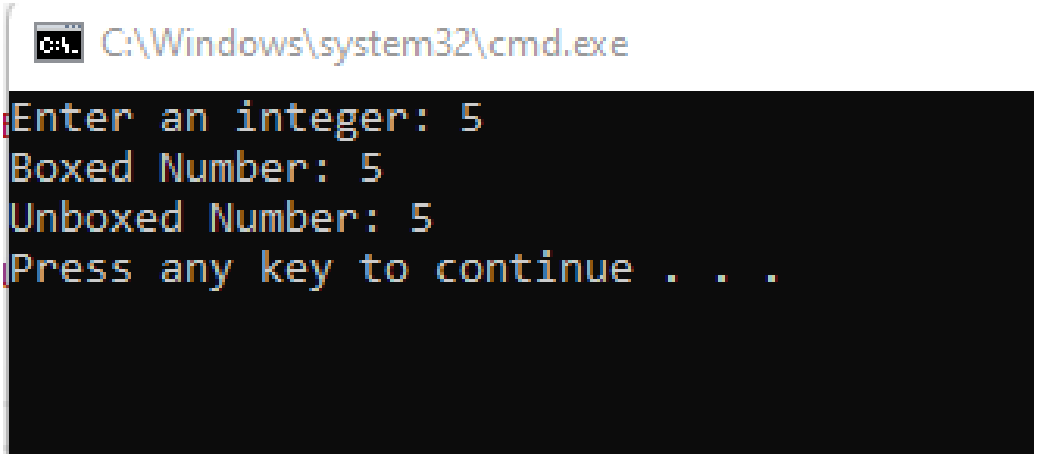
**1. Programs to demonstrate Boxing and Unboxing**

```
using System;

class box
{
   static void Main(string[] args)
   {
      Console.Write("Enter an integer: ");
      int n = Convert.ToInt32(Console.ReadLine());

      object a = n;
      Console.WriteLine("Boxed Number: " + a);

      int b = (int)a;
      Console.WriteLine("Unboxed Number: " + b);
   }
}
```

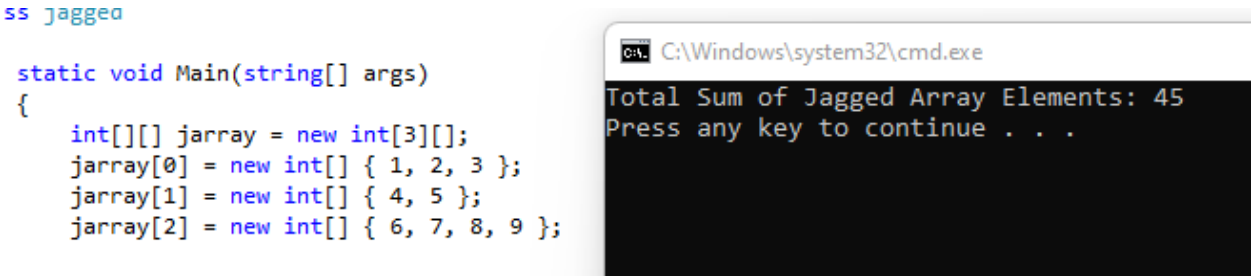



**2. Program to demonstrate the sum of all the elements present in a jagged array of 3 inner arrays.**

```
using System;
```

```
class jagged
{
   static void Main(string[] args)
   {
      int[][] jarray = new int[3][];
      jarray[0] = new int[] { 1, 2, 3 };
      jarray[1] = new int[] { 4, 5 };
      jarray[2] = new int[] { 6, 7, 8, 9 };


      int total = 0;
      foreach (var i in jarray)
      {
         foreach (var j in i)
         {
            total += j;
         }
      }

      Console.WriteLine("Total Sum of Jagged Array Elements: " + total);
   }
}
```

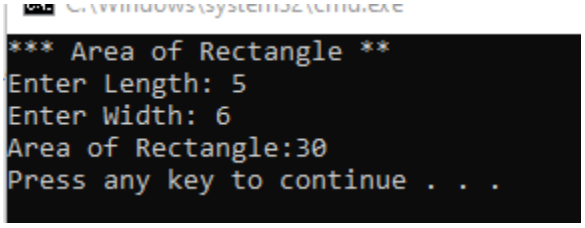
OUTPUT:




**3. Programs to demonstrate Classes and Objects.**

```
using System;

class Rect
{
```

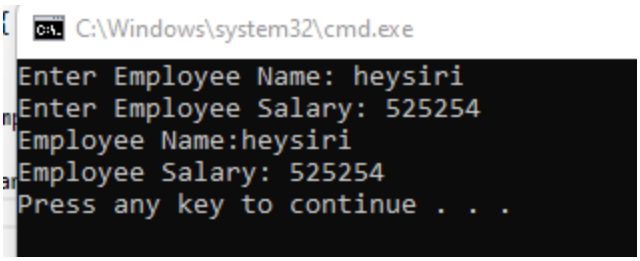
```
    private double len;
    private double wid;

    public Rect(double length, double width)
    {
        len = length;
        wid = width;
    }

    public double GetArea()
    {
        return len * wid;
    }
}

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("*** Area of Rectangle **");
        Console.Write("Enter Length: ");
        double length = Convert.ToDouble(Console.ReadLine());
        Console.Write("Enter Width: ");
        double width = Convert.ToDouble(Console.ReadLine());
        Rect r = new Rect(length, width);
        Console.WriteLine("Area of Rectangle:" + r.GetArea());
    }
}
```




4. Programs to illustrate the use of different properties in C#

```
using System;
```

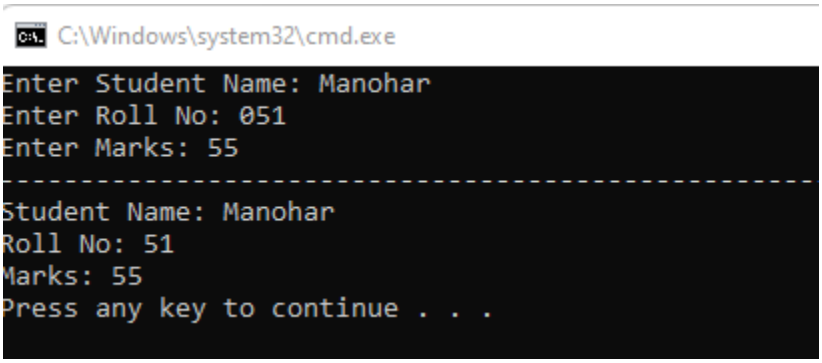
```
class Emp
{
   private string empName;
   private decimal empSalary;

   public string Name
   {
      get { return empName; }
      set { empSalary = value; }
   }

   public Emp(string name)
   {
      empName = name;
   }

   public void DisplayInfo()
   {
      Console.WriteLine("Employee Name:" +  Name);
      Console.WriteLine("Employee Salary: " +  empSalary );
   }
}

class Pro
{
   static void Main(string[] args)
   {
      Console.Write("Enter Employee Name: ");
      string name = Console.ReadLine();
      Employee emp = new Employee(name);
      Console.Write("Enter Employee Salary: ");
      decimal salary = Convert.ToDecimal(Console.ReadLine());
      emp.Salary = salary;
      emp.DisplayInfo();
   }}
```




**5. Programs to demonstrate Exploring Structs.**

```
using System;

struct Student
{
    public string name;
    public int rollNo;
    public double marks;

    public void Display()
    {
        Console.WriteLine("Student Name: " + name);
        Console.WriteLine("Roll No: " + rollNo);
        Console.WriteLine("Marks: " + marks);
    }
}

class Prog
{
    static void Main()
    {
        Student s1;


        Console.Write("Enter Student Name: ");
        s1.name = Console.ReadLine();

        Console.Write("Enter Roll No: ");
        s1.rollNo = int.Parse(Console.ReadLine());
```

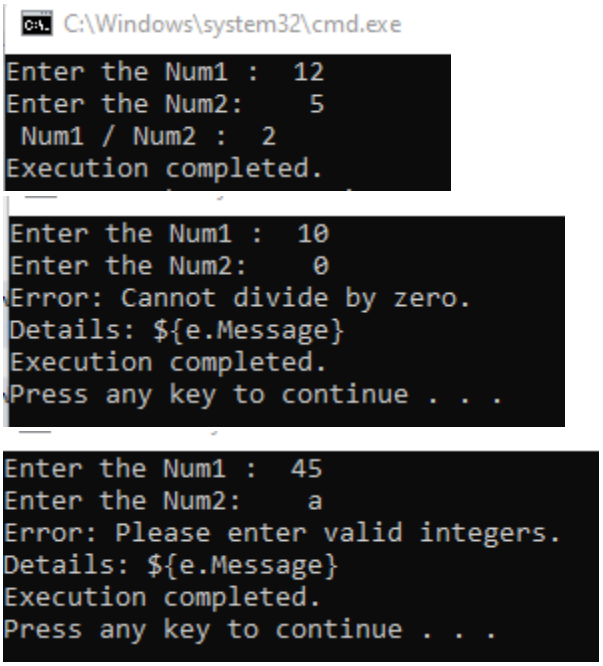
```
        Console.Write("Enter Marks: ");
        s1.marks = double.Parse(Console.ReadLine());
        Console.WriteLine("------------------------------------------------------------------------");
        s1.Display();

    }
}
```



**6. Programs to demonstrate Exception Handling.**

```
using System;

class Program
{
   static void Main(string[] args)
   {
      try
      {

         Console.Write("Enter the Num1 :  ");
         int num1 = int.Parse(Console.ReadLine());

         Console.Write("Enter the Num2:    ");
         int num2 = int.Parse(Console.ReadLine());



         int result =num1 / num2;
         Console.WriteLine(" Num1 / Num2 :  " + result);
      }
      catch (DivideByZeroException e)
      {
         Console.WriteLine("Error: Cannot divide by zero.");
         Console.WriteLine("Details: ${e.Message}");
      }
      catch (FormatException e)
      {
         Console.WriteLine("Error: Please enter valid integers.");
         Console.WriteLine("Details: ${e.Message}");
      }
      catch (Exception e)
      {
         Console.WriteLine("An unexpected error occurred.");
         Console.WriteLine("Details: ${e.Message}");
      }
      finally
      {
```

```
            Console.WriteLine("Execution completed.");
        }
    }
}
```

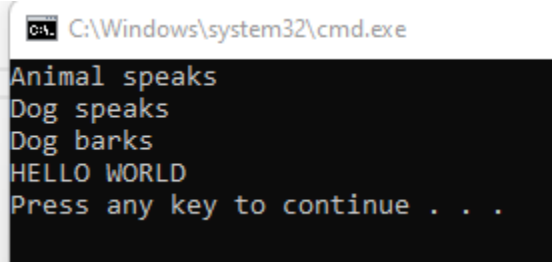



**7. Program to demonstrate inheritance covering the concepts of Sealed Classes,**

**Sealed Methods,Extension Methods.**

```
using System;

class Animal
{
    public string name;

    public void Speak()
    {
        Console.WriteLine("Animal speaks");
    }

    public sealed void Move()
    {
        Console.WriteLine("Animal moves");
    }
}

sealed class Dog : Animal
{
    public void Bark()
    {
        Console.WriteLine("Dog barks");
    }

    public new void Speak()
    {
        Console.WriteLine("Dog speaks");
    }
}


static class StringExtensions
{
    public static void PrintUpperCase(this string str)
    {
        Console.WriteLine(str.ToUpper());
    }
}
```

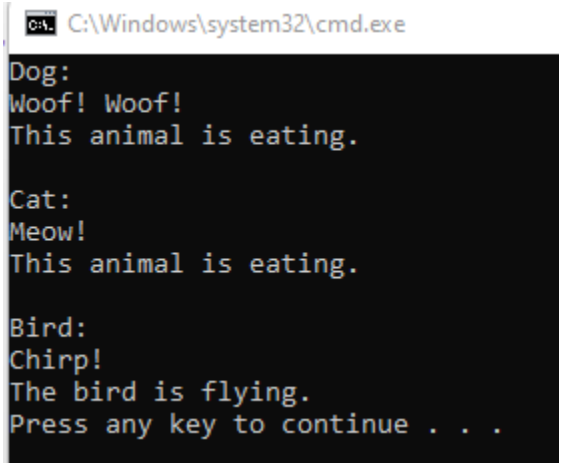
```
class Program
{
   static void Main()
   {

      Animal animal = new Animal();
      animal.name = "Generic Animal";
      animal.Speak();
      animal.Move();

      Dog dog = new Dog();
      dog.name = "Buddy";
      dog.Speak();
      dog.Move();
      dog.Bark();


      string message = "hello world";
      message.PrintUpperCase();
   }
}
```




**8. Programs to demonstrate Abstract Classes and Interfaces.**

```
using System;

abstract class Animal
{
    public abstract void Speak();

    public void Eat()
    {
        Console.WriteLine("This animal is eating.");
    }
}

class Dog : Animal
{
    public override void Speak()
    {
        Console.WriteLine("Woof! Woof!");
    }
}

class Cat : Animal
{
    public override void Speak()
    {
        Console.WriteLine("Meow!");
    }
}


interface IMovable
{
    void Move();
}

class Bird : Animal, IMovable
{
    public override void Speak()
    {
        Console.WriteLine("Chirp!");
```

```
    }

    public void Move()
    {
        Console.WriteLine("The bird is flying.");
    }
}

class Pro
{
    static void Main(string[] args)
    {
        Animal dog = new Dog();
        Animal cat = new Cat();
        Bird bird = new Bird();

        Console.WriteLine("Dog:");
        dog.Speak();
        dog.Eat();

        Console.WriteLine("\nCat:");
        cat.Speak();
        cat.Eat();
        Console.WriteLine("\nBird:");
        bird.Speak();
        bird.Move();
    }
}
```

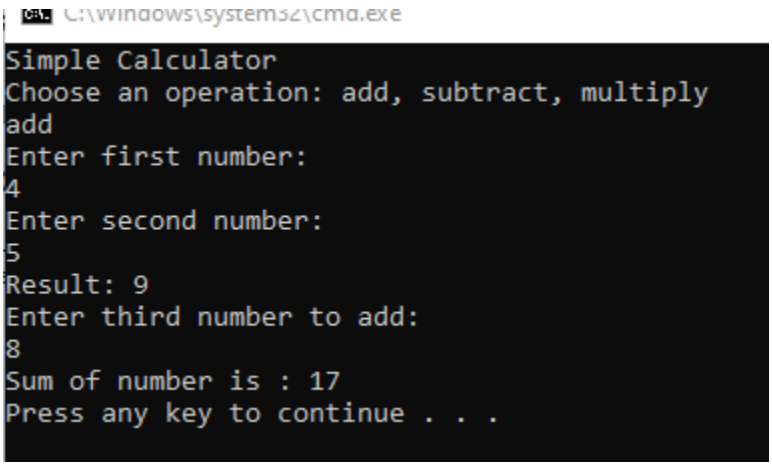


9. Programs to demonstrate Polymorphism covering the concepts of Overloading, Overriding, Virtual

```
using System;

public class Cal
{
    public virtual double Calculate(double a, double b)
    {
        return 0;
    }
}

public class AddCalculator : Cal
{
    public override double Calculate(double a, double b)
    {
        return a + b;
    }
}

public class SubtractCalculator : Cal
{
    public override double Calculate(double a, double b)
    {
        return a - b;
    }
}

public class MultiplyCalculator : Cal
{
    public override double Calculate(double a, double b)
    {
        return a * b;
    }
}
```

```
class Pro
{
   static void Main(string[] args)
   {
      Console.WriteLine("Simple Calculator");
      Console.WriteLine("Choose an operation: add, subtract, multiply");
      string operation = Console.ReadLine().ToLower();

      Console.WriteLine("Enter first number:");
      double num1 = Convert.ToDouble(Console.ReadLine());

      Console.WriteLine("Enter second number:");
      double num2 = Convert.ToDouble(Console.ReadLine());

      Calculator calculator;

      switch (operation)
      {
         case "add":
            calculator = new AddCalculator();
            break;
         case "subtract":
            calculator = new SubtractCalculator();
            break;
         case "multiply":
            calculator = new MultiplyCalculator();
            break;
         default:
            Console.WriteLine("Invalid operation.");
            return;
      }

      double result = calculator.Calculate(num1, num2);
      Console.WriteLine("Result: {0}",result);

      AdvancedCalculator advancedCalc = new AdvancedCalculator();
      Console.WriteLine("Enter third number to add:");
      double num3 = Convert.ToDouble(Console.ReadLine());

   }
```

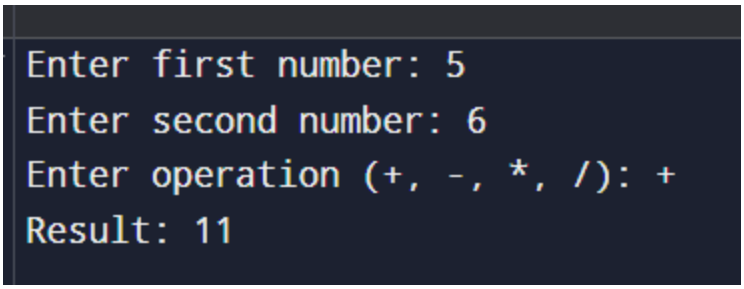
}

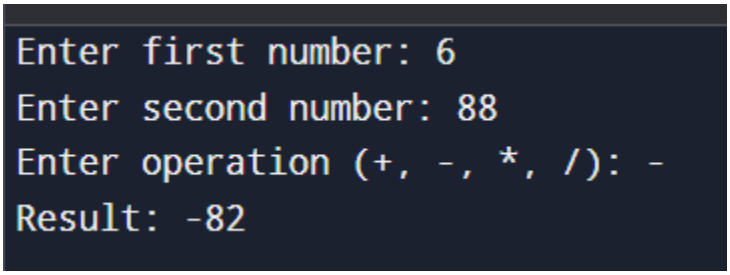



**10. Programs to demonstrate on Operator Overloading.**

```
using System;

class Num{
    public int val;

    public Number(int v) { val = v; }

    public static Number operator +(Number n1, Number n2) {
        return new Number(n1.val + n2.val);
    }

    public static Number operator -(Number n1, Number n2) {
        return new Number(n1.val - n2.val);
    }

    public static Number operator *(Number n1, Number n2) {
        return new Number(n1.val * n2.val);
    }

    public static Number operator /(Number n1, Number n2) {
```

```
        return new Number(n2.val != 0 ? n1.val / n2.val : 0);
    }

    public void Show(string msg) {
        Console.WriteLine(msg + val);
    }
}

class Program {
    static void Main() {
        Console.Write("Enter first number: ");
        int a = Convert.ToInt32(Console.ReadLine());

        Console.Write("Enter second number: ");
        int b = Convert.ToInt32(Console.ReadLine());

        Console.Write("Enter operation (+, -, *, /): ");
        char op = Convert.ToChar(Console.ReadLine());

        Number n1 = new Number(a);
        Number n2 = new Number(b);
        Number result;

        switch (op) {
            case '+': result = n1 + n2; result.Show("Result: "); break;
            case '-': result = n1 - n2; result.Show("Result: "); break;
            case '*': result = n1 * n2; result.Show("Result: "); break;
            case '/':
                if (n2.val == 0) Console.WriteLine("Cannot divide by zero!");
                else { result = n1 / n2; result.Show("Result: "); }
                break;
            default: Console.WriteLine("Invalid operation!"); break;
        }
    }
}
```




**11. Programs to demonstrate Delegates and Event Handlers.**

```
using System;
using System.Threading;

public delegate void WorkPerfor(int hours, string workType);
public delegate void WorkComtd(string workType);

public class Worker
{
    public event WorkPerfor WorkPerformed;
    public event WorkComtd WorkCompleted;

    public void DoWork(int hours, string workType)
    {
        for (int i = 0; i < hours; i++)
        {
            // Simulating work
            Thread.Sleep(1000);
            WorkPerformed?.Invoke(i + 1, workType);
        }
        WorkCompleted?.Invoke(workType);
```

```
    }
}

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Enter the number of hours to work:");
        int hours = Convert.ToInt32(Console.ReadLine());

        Worker worker = new Worker();

        // Subscribing to events
        worker.WorkPerformed += Worker_WorkPerformed;
        worker.WorkCompleted += Worker_WorkCompleted;

        worker.DoWork(hours, "Generating Report");

        Console.ReadKey();
    }

    static void Worker_WorkPerformed(int hours, string workType)
    {
        Console.WriteLine($"{hours} hour(s) completed for {workType}");
    }

    static void Worker_WorkCompleted(string workType)
    {
        Console.WriteLine($"{workType} work completed!");
    }
}
```

```
Enter the number of hours to work:
10
1 hour(s) completed for Generating Report
2 hour(s) completed for Generating Report
3 hour(s) completed for Generating Report
4 hour(s) completed for Generating Report
5 hour(s) completed for Generating Report
6 hour(s) completed for Generating Report
7 hour(s) completed for Generating Report
8 hour(s) completed for Generating Report
9 hour(s) completed for Generating Report
10 hour(s) completed for Generating Report
Generating Report work completed!
```

**12. Programs to demonstrate on System collections.**

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main(string[] args)
    {
        List<string> names = new List<string>();
        string input;

        Console.WriteLine("Enter names (type 'exit' to finish):");

        while (true)
        {
            input = Console.ReadLine();
            if (input.ToLower() == "exit")
                break;

            names.Add(input);
        }

        Console.WriteLine("\nNames entered:");
        foreach (var name in names)
        {
            Console.WriteLine(name);
        }

        Console.WriteLine("\nEnter a name to remove:");
        string nameToRemove = Console.ReadLine();

        if (names.Remove(nameToRemove))
        {
            Console.WriteLine($"{nameToRemove} has been removed.");
        }
        else
        {
            Console.WriteLine($"{nameToRemove} not found in the list.");
        }
```

```
        Console.WriteLine("\nFinal list of names:");
        foreach (var name in names)
        {
            Console.WriteLine(name);
        }

        Console.ReadKey();
    }
}
```

```
Enter names (type 'exit' to finish):
manu
Manohar
Bhaskar
exit

Names entered:
manu
Manohar
Bhaskar

Enter a name to remove:
manu
manu has been removed.

Final list of names:
Manohar
Bhaskar
```